

MongoDB Commands

R.Jagadeesh

2520090030

The screenshot displays the OneCompiler MongoDB Online Editor interface. The left pane shows a JavaScript script for interacting with a MongoDB database. The script includes commands for inserting a document, counting documents, inserting multiple documents, and performing various find operations. The right pane shows the output of the script, which is a JSON array of documents. The output includes the inserted document, the result of the countDocuments command, and the results of the find operations.

```
14 })
15 db.students.find()
16 db.students.find().forEach(printjson)
17 db.students.insertOne({
18   rollNo: 111,
19   name: "Vijay",
20   age: 21,
21   department: "CSE",
22   marks: 86,
23   city: "Hyderabad"
24 })
25 db.students.countDocuments()
26 db.students.insertMany([
27   { rollNo: 112, name: "Pooja", age: 20, department: "IT", marks: 84 },
28   { rollNo: 113, name: "Ramesh", age: 22, department: "ECE", marks: 79 }
29 ])
30 db.students.countDocuments()
31 db.students.find()
32 db.students.findOne()
33 db.students.findOne({ rollNo: 101 })
34 db.students.find({ name: "John" })
35 db.students.countDocuments()
36 db.students.countDocuments({ department: "CSE" })
37 db.students.find({ marks: { $gt: 80 } })
38 db.students.find({ marks: { $gte: 80 } })
39 db.students.find({ marks: { $lt: 80 } })
40
```

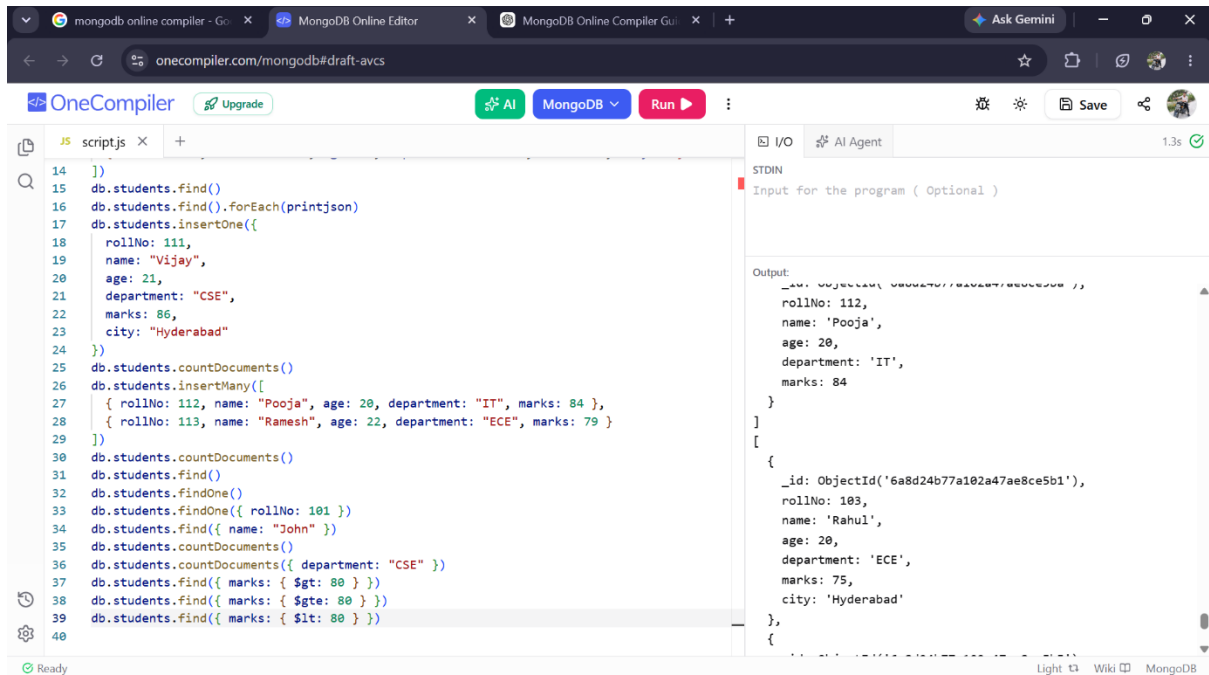
Output:

```
{
  "_id": ObjectId('6a8d24b77a102a47ae8ce5b5'),
  "rollNo": 107,
  "name": "Ravi",
  "age": 23,
  "department": "ECE",
  "marks": 68,
  "city": "Nellore"
},
{
  "_id": ObjectId('6a8d24b77a102a47ae8ce5b1'),
  "rollNo": 113,
  "name": "Ramesh",
  "age": 22,
  "department": "ECE",
  "marks": 79
}
```

MongoDB Commands

R.Jagadeesh

2520090030

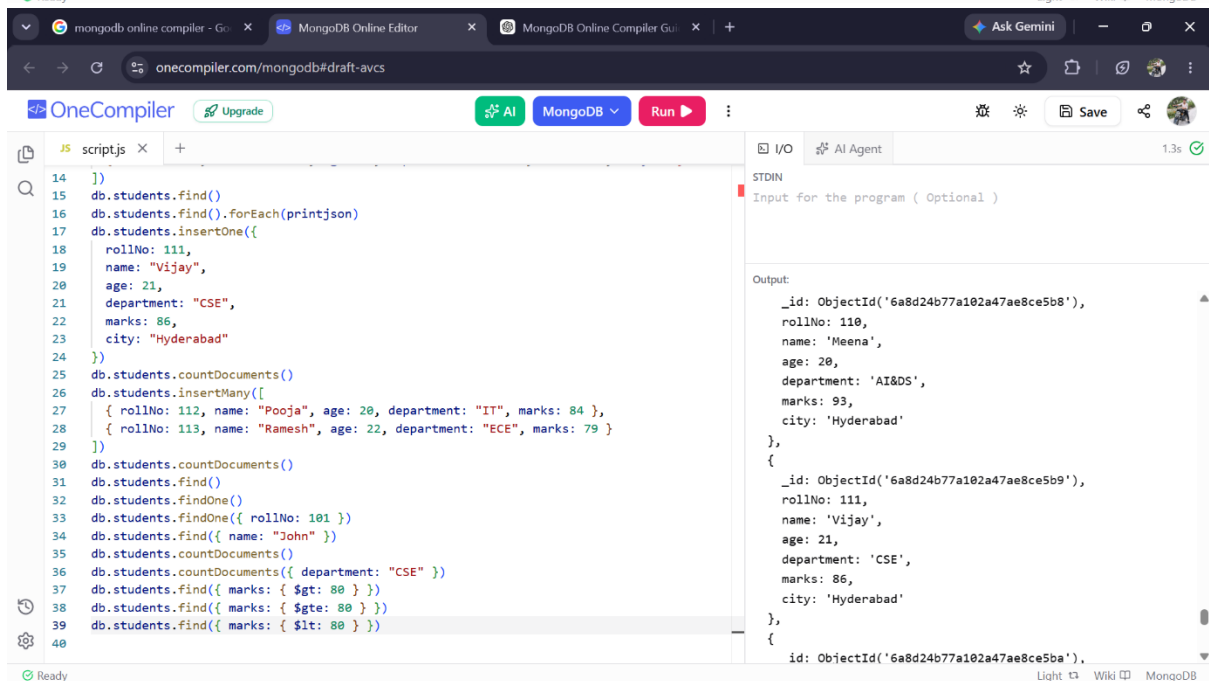


The screenshot shows the OneCompiler MongoDB Online Editor interface. The left pane contains a JavaScript script for interacting with a MongoDB database. The script performs the following operations:

- Inserts a document into the `students` collection: `{ rollNo: 111, name: "Vijay", age: 21, department: "CSE", marks: 86, city: "Hyderabad" }`.
- Counts the number of documents in the `students` collection.
- Inserts multiple documents into the `students` collection: `{ rollNo: 112, name: "Pooja", age: 20, department: "IT", marks: 84 }, { rollNo: 113, name: "Ramesh", age: 22, department: "ECE", marks: 79 }`.
- Counts the number of documents in the `students` collection.
- Finds all documents in the `students` collection and prints them to the console.
- Finds a document with `rollNo: 101`.
- Finds a document with `name: "John"`.
- Counts the number of documents in the `students` collection.
- Counts the number of documents in the `students` collection where the department is "CSE".
- Finds documents where marks are greater than 80.
- Finds documents where marks are greater than or equal to 80.
- Finds documents where marks are less than 80.

The right pane shows the output of the script, which is a JSON array of documents:

```
[{"_id": "6a8d24b77a102a47ae8ce5b1", "rollNo": 112, "name": "Pooja", "age": 20, "department": "IT", "marks": 84}, {"_id": "6a8d24b77a102a47ae8ce5b2", "rollNo": 113, "name": "Ramesh", "age": 22, "department": "ECE", "marks": 79}, {"_id": "6a8d24b77a102a47ae8ce5b3", "rollNo": 111, "name": "Vijay", "age": 21, "department": "CSE", "marks": 86, "city": "Hyderabad"}]
```



The screenshot shows the OneCompiler MongoDB Online Editor interface. The left pane contains a JavaScript script for interacting with a MongoDB database. The script performs the following operations:

- Inserts a document into the `students` collection: `{ rollNo: 111, name: "Vijay", age: 21, department: "CSE", marks: 86, city: "Hyderabad" }`.
- Counts the number of documents in the `students` collection.
- Inserts multiple documents into the `students` collection: `{ rollNo: 112, name: "Pooja", age: 20, department: "IT", marks: 84 }, { rollNo: 113, name: "Ramesh", age: 22, department: "ECE", marks: 79 }`.
- Counts the number of documents in the `students` collection.
- Finds all documents in the `students` collection and prints them to the console.
- Finds a document with `rollNo: 101`.
- Finds a document with `name: "John"`.
- Counts the number of documents in the `students` collection.
- Counts the number of documents in the `students` collection where the department is "CSE".
- Finds documents where marks are greater than 80.
- Finds documents where marks are greater than or equal to 80.
- Finds documents where marks are less than 80.

The right pane shows the output of the script, which is a JSON array of documents:

```
[{"_id": "6a8d24b77a102a47ae8ce5b8", "rollNo": 110, "name": "Meena", "age": 20, "department": "AI&DS", "marks": 93, "city": "Hyderabad"}, {"_id": "6a8d24b77a102a47ae8ce5b9", "rollNo": 111, "name": "Vijay", "age": 21, "department": "CSE", "marks": 86, "city": "Hyderabad"}, {"_id": "6a8d24b77a102a47ae8ce5ba", "rollNo": 112, "name": "Pooja", "age": 20, "department": "IT", "marks": 84, "city": "Hyderabad"}, {"_id": "6a8d24b77a102a47ae8ce5bb", "rollNo": 113, "name": "Ramesh", "age": 22, "department": "ECE", "marks": 79, "city": "Hyderabad"}]
```

MongoDB Commands

R.Jagadeesh

2520090030

The screenshot displays the OneCompiler MongoDB Online Editor interface. The left pane shows a JavaScript script for interacting with a MongoDB database. The script performs the following operations:

- Find all documents in the `students` collection and print them as JSON.
- Insert a new document: `{ rollNo: 111, name: "Vijay", age: 21, department: "CSE", marks: 86, city: "Hyderabad" }`.
- Count the number of documents in the `students` collection.
- Insert multiple documents: `{ rollNo: 112, name: "Pooja", age: 20, department: "IT", marks: 84 }` and `{ rollNo: 113, name: "Ramesh", age: 22, department: "ECE", marks: 79 }`.
- Count the number of documents in the `students` collection.
- Find a document by `rollNo: 101`.
- Find a document by `name: "John"`.
- Count the number of documents in the `students` collection.
- Count the number of documents where the `department` is "CSE".
- Find documents where `marks` is greater than or equal to 80.
- Find documents where `marks` is less than or equal to 80.

The right pane shows the output of the script, which is a JSON array of documents:

```
[{"_id": "6a8d24b77a102a47ae8ce5b7", "rollNo": 108, "name": "Sita", "age": 21, "department": "CSE", "marks": 91, "city": "Hyderabad"}, {"_id": "6a8d24b77a102a47ae8ce5b8", "rollNo": 109, "name": "Arun", "age": 22, "department": "AI&DS", "marks": 87, "city": "Kurnool"}, {"_id": "6a8d24b77a102a47ae8ce5b9", "rollNo": 110, "name": "Vijay", "age": 21, "department": "CSE", "marks": 86, "city": "Hyderabad"}, {"_id": "6a8d24b77a102a47ae8ce5ba", "rollNo": 112, "name": "Pooja", "age": 20, "department": "IT", "marks": 84, "city": "Hyderabad"}, {"_id": "6a8d24b77a102a47ae8ce5bb", "rollNo": 113, "name": "Ramesh", "age": 22, "department": "ECE", "marks": 79, "city": "Hyderabad"}]
```

MongoDB Commands

R.Jagadeesh

2520090030

The screenshot displays the OneCompiler MongoDB Online Editor interface. The left pane shows a JavaScript script for interacting with a MongoDB database. The script performs the following operations:

- Inserts a document into the `students` collection with `rollNo: 111`, `name: "Vijay"`, `age: 21`, `department: "CSE"`, `marks: 86`, and `city: "Hyderabad"`.
- Counts the documents in the `students` collection.
- Inserts multiple documents into the `students` collection: `{ rollNo: 112, name: "Pooja", age: 20, department: "IT", marks: 84 }` and `{ rollNo: 113, name: "Ramesh", age: 22, department: "ECE", marks: 79 }`.
- Counts the documents in the `students` collection.
- Retrieves all documents from the `students` collection using `db.students.find()`.
- Retrieves a single document from the `students` collection with `rollNo: 101` using `db.students.findOne({ rollNo: 101 })`.
- Retrieves a single document from the `students` collection with `name: "John"` using `db.students.find({ name: "John" })`.
- Counts the documents in the `students` collection.
- Retrieves documents from the `students` collection where the department is "CSE" using `db.students.countDocuments({ department: "CSE" })`.
- Retrieves documents from the `students` collection where marks are greater than or equal to 80 using `db.students.find({ marks: { $gt: 80 } })`.
- Retrieves documents from the `students` collection where marks are greater than or equal to 80 using `db.students.find({ marks: { $gte: 80 } })`.
- Retrieves documents from the `students` collection where marks are less than or equal to 80 using `db.students.find({ marks: { $lte: 80 } })`.

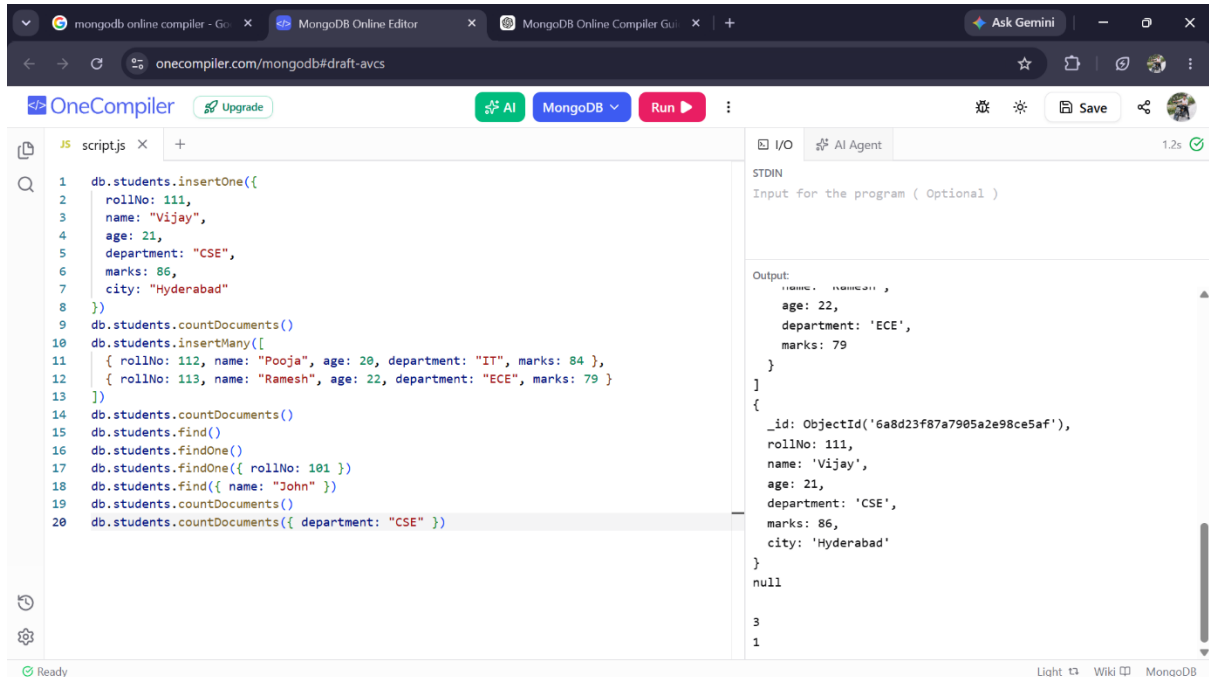
The right pane shows the output of the script, which is a JSON array of documents:

```
{
  "_id": ObjectId('6a8d24b77a102a47ae8ce5b8'),
  "rollNo": 112,
  "name": 'Pooja',
  "age": 20,
  "department": 'IT',
  "marks": 84
},
{
  "_id": ObjectId('6a8d24b77a102a47ae8ce5b1'),
  "rollNo": 103,
  "name": 'Rahul',
  "age": 20,
  "department": 'ECE',
  "marks": 75,
  "city": 'Hyderabad'
},
{
  "_id": ObjectId('6a8d24b77a102a47ae8ce5b9'),
  "rollNo": 111,
  "name": 'Vijay',
  "age": 21,
  "department": 'CSE',
  "marks": 86,
  "city": 'Hyderabad'
},
{
  "_id": ObjectId('6a8d24b77a102a47ae8ce5b1'),
  "rollNo": 103,
  "name": 'Rahul',
  "age": 20,
  "department": 'ECE',
  "marks": 75,
  "city": 'Hyderabad'
}
```

MongoDB Commands

R.Jagadeesh

2520090030

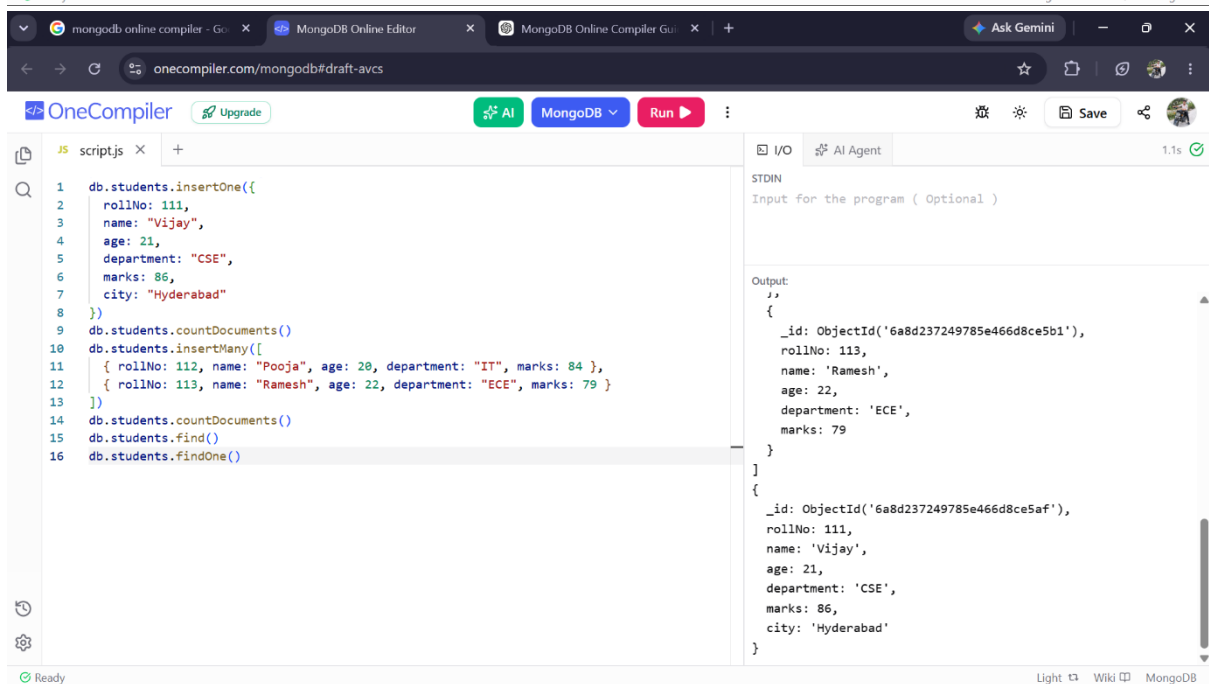


The screenshot shows the OneCompiler MongoDB Online Editor interface. The left pane contains a JavaScript script for interacting with a MongoDB database. The right pane shows the output of the script, which includes the results of insert, count, and find operations.

```
1 db.students.insertOne({
2   rollNo: 111,
3   name: "Vijay",
4   age: 21,
5   department: "CSE",
6   marks: 86,
7   city: "Hyderabad"
8 })
9 db.students.countDocuments()
10 db.students.insertMany([
11   { rollNo: 112, name: "Pooja", age: 20, department: "IT", marks: 84 },
12   { rollNo: 113, name: "Ramesh", age: 22, department: "ECE", marks: 79 }
13 ])
14 db.students.countDocuments()
15 db.students.find()
16 db.students.findOne()
17 db.students.findOne({ rollNo: 101 })
18 db.students.find({ name: "John" })
19 db.students.countDocuments()
20 db.students.countDocuments({ department: "CSE" })
```

Output:

```
{
  age: 22,
  department: 'ECE',
  marks: 79
}
{
  _id: ObjectId('6a8d23f87a7905a2e98ce5af'),
  rollNo: 111,
  name: 'Vijay',
  age: 21,
  department: 'CSE',
  marks: 86,
  city: 'Hyderabad'
}
null
3
1
```



The screenshot shows the OneCompiler MongoDB Online Editor interface. The left pane contains a JavaScript script for interacting with a MongoDB database. The right pane shows the output of the script, which includes the results of insert, count, and find operations.

```
1 db.students.insertOne({
2   rollNo: 111,
3   name: "Vijay",
4   age: 21,
5   department: "CSE",
6   marks: 86,
7   city: "Hyderabad"
8 })
9 db.students.countDocuments()
10 db.students.insertMany([
11   { rollNo: 112, name: "Pooja", age: 20, department: "IT", marks: 84 },
12   { rollNo: 113, name: "Ramesh", age: 22, department: "ECE", marks: 79 }
13 ])
14 db.students.countDocuments()
15 db.students.find()
16 db.students.findOne()
```

Output:

```
{
  _id: ObjectId('6a8d237249785e466d8ce5b1'),
  rollNo: 113,
  name: 'Ramesh',
  age: 22,
  department: 'ECE',
  marks: 79
}
{
  _id: ObjectId('6a8d237249785e466d8ce5af'),
  rollNo: 111,
  name: 'Vijay',
  age: 21,
  department: 'CSE',
  marks: 86,
  city: 'Hyderabad'
}
```

MongoDB Commands

R.Jagadeesh

2520090030

Ask Gemini

—

×

☆

⚙️

☀️

Save

I/O

AI Agent

1.1s ✓

ITDIN

Input for the program (Optional)

Output:

```
City: Hyderabad
},
{
  _id: ObjectId('6a8d22fc4fcd0c4fd68ce5b0'),
  rollNo: 112,
  name: 'Pooja',
  age: 20,
  department: 'IT',
  marks: 84
},
{
  _id: ObjectId('6a8d22fc4fcd0c4fd68ce5b1'),
  rollNo: 113,
  name: 'Ramesh',
  age: 22,
  department: 'ECE',
  marks: 79
}
|
```

Light ↺ Wiki 📖 MongoDB

MongoDB Commands

R.Jagadeesh

2520090030

Ask Gemini

—

×

☆

🔧

☀️

📄 Save

🔗

I/O

AI Agent

1.1s ✓

STDIN

Input for the program (Optional)

Output:

```
{
  _id: ObjectId('6a8d22fc4fcd0c4fd68ce5af'),
  rollNo: 111,
  name: 'Vijay',
  age: 21,
  department: 'CSE',
  marks: 86,
  city: 'Hyderabad'
},
{
  _id: ObjectId('6a8d22fc4fcd0c4fd68ce5b0'),
  rollNo: 112,
  name: 'Pooja',
  age: 20,
  department: 'IT',
  marks: 84
},
{
  _id: ObjectId('6a8d22fc4fcd0c4fd68ce5b1'),
  rollNo: 113,
  name: 'Ravi',
  age: 22,
  department: 'CSE',
  marks: 88,
  city: 'Hyderabad'
}
```

Light 🔁 Wiki 📖 MongoDB

MongoDB Commands

R.Jagadeesh

2520090030

The screenshot shows the OneCompiler MongoDB Online Editor interface. The left pane contains a JavaScript script for interacting with a MongoDB database. The script performs the following actions:

- Inserts a document into the `students` collection.
- Counts the number of documents in the `students` collection.
- Inserts multiple documents into the `students` collection.
- Counts the number of documents in the `students` collection again.
- Finds all documents in the `students` collection.

The right pane displays the output of the script, which is a JSON object representing the result of the `find` operation. The output shows three documents in the `students` collection.

```
1 db.students.insertOne({
2   rollNo: 111,
3   name: "Vijay",
4   age: 21,
5   department: "CSE",
6   marks: 86,
7   city: "Hyderabad"
8 })
9 db.students.countDocuments()
10 db.students.insertMany([
11   { rollNo: 112, name: "Pooja", age: 20, department: "IT", marks: 84 },
12   { rollNo: 113, name: "Ramesh", age: 22, department: "ECE", marks: 79 }
13 ])
14 db.students.countDocuments()
15 db.students.find()
```

Output:

```
{
  acknowledged: true,
  insertedId: ObjectId('6a8d22fc4fcd0c4fd68ce5af')
}
1
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a8d22fc4fcd0c4fd68ce5b0'),
    '1': ObjectId('6a8d22fc4fcd0c4fd68ce5b1')
  }
}
3
[
  {
    _id: ObjectId('6a8d22fc4fcd0c4fd68ce5af'),
    rollNo: 111,
    name: 'Vijay',
    age: 21,
```

The screenshot shows the OneCompiler MongoDB Online Editor interface. The left pane contains a JavaScript script for interacting with a MongoDB database. The script performs the following actions:

- Inserts a document into the `students` collection.

The right pane displays the output of the script, which is a JSON object representing the result of the `insertOne` operation. The output shows that the document was successfully inserted into the `students` collection.

```
1 db.students.insertOne({
2   rollNo: 111,
3   name: "Vijay",
4   age: 21,
5   department: "CSE",
6   marks: 86,
7   city: "Hyderabad"
8 })
```

Output:

```
{
  acknowledged: true,
  insertedId: ObjectId('6a8d22961e1dc8e3cc8ce5af')
}
```

MongoDB Commands

R.Jagadeesh

2520090030

I/O

AI Agent

1.7s 

STDIN

Input for the program (Optional)

Output:

```
{
  _id: ObjectId('6a8d221c676b5f4d9f8ce5b7'),
  rollNo: 109,
  name: 'Arun',
  age: 22,
  department: 'AI&DS',
  marks: 87,
  city: 'Kurnool'
}
{
  _id: ObjectId('6a8d221c676b5f4d9f8ce5b8'),
  rollNo: 110,
  name: 'Meena',
  age: 20,
  department: 'AI&DS',
  marks: 93,
  city: 'Hyderabad'
}
```

Light  Wiki  MongoDB

MongoDB Commands

R.Jagadeesh

2520090030

I/O

AI Agent

1.7s 

STDIN

Input for the program (Optional)

Output:

```
    age: 20,
    department: 'ECE',
    marks: 75,
    city: 'Hyderabad'
  }
  {
    _id: ObjectId('6a8d221c676b5f4d9f8ce5b2'),
    rollNo: 104,
    name: 'Priya',
    age: 22,
    department: 'IT',
    marks: 88,
    city: 'Warangal'
  }
  {
    _id: ObjectId('6a8d221c676b5f4d9f8ce5b3'),
    rollNo: 105,
    name: 'Kiran',
    age: 21,
```

Light  Wiki  MongoDB

MongoDB Commands

R.Jagadeesh

2520090030



Save



I/O

AI Agent

1.7s



STDIN

Input for the program (Optional)

Output:

```
{
  _id: ObjectId('6a8d221c676b5f4d9f8ce5b6'),
  rollNo: 108,
  name: 'Sita',
  age: 21,
  department: 'CSE',
  marks: 91,
  city: 'Hyderabad'
},
{
  _id: ObjectId('6a8d221c676b5f4d9f8ce5b7'),
  rollNo: 109,
  name: 'Arun',
  age: 22,
  department: 'AI&DS',
  marks: 87,
  city: 'Kurnool'
},
{
```

Light

Wiki

MongoDB

MongoDB Commands

R.Jagadeesh

2520090030

Output:

```
-----, ----- ,
  rollNo: 105,
  name: 'Kiran',
  age: 21,
  department: 'CSE',
  marks: 95,
  city: 'Hyderabad'
},
{
  _id: ObjectId('6a8d221c676b5f4d9f8ce5b4'),
  rollNo: 106,
  name: 'Anu',
  age: 20,
  department: 'IT',
  marks: 82,
  city: 'Guntur'
},
{
  _id: ObjectId('6a8d221c676b5f4d9f8ce5b5'),
  rollNo: 107,
```

MongoDB Commands

R.Jagadeesh

2520090030

I/O

AI Agent

1.7s 

STDIN

Input for the program (Optional)

Output:

```
_id: ObjectId('6a8d221c676b5f4d9f8ce5b1'),
rollNo: 103,
name: 'Rahul',
age: 20,
department: 'ECE',
marks: 75,
city: 'Hyderabad'
},
{
  _id: ObjectId('6a8d221c676b5f4d9f8ce5b2'),
  rollNo: 104,
  name: 'Priya',
  age: 22,
  department: 'IT',
  marks: 88,
  city: 'Warangal'
},
{
  _id: ObjectId('6a8d221c676b5f4d9f8ce5b3'),
```

Light  Wiki  MongoDB

MongoDB Commands

R.Jagadeesh

2520090030

I/O

AI Agent

1.7s 

STDIN

Input for the program (Optional)

Output:

```
[
  {
    _id: ObjectId('6a8d221c676b5f4d9f8ce5af'),
    rollNo: 101,
    name: 'John',
    age: 20,
    department: 'CSE',
    marks: 85,
    city: 'Hyderabad'
  },
  {
    _id: ObjectId('6a8d221c676b5f4d9f8ce5b0'),
    rollNo: 102,
    name: 'Alice',
    age: 21,
    department: 'CSE',
    marks: 90,
    city: 'Vijayawada'
  }
]
```

Light  Wiki  MongoDB

MongoDB Commands

R.Jagadeesh

2520090030

The image displays two screenshots of the OneCompiler MongoDB Online Editor interface, showing the execution of MongoDB commands.

Top Screenshot:

- Code Editor:** Contains a JavaScript script to create a collection named 'students' and insert 10 documents. The documents represent students with fields like rollNo, name, age, department, marks, and city.
- Output:** Shows the result of the execution, indicating that 10 documents were successfully inserted into the 'students' collection. The output includes the 'ok' status and the 'insertedIds' for each document.

Bottom Screenshot:

- Code Editor:** Contains a JavaScript script to switch to the 'studentDB' database and show the databases.
- Output:** Shows the result of the execution, indicating that the database has been switched to 'studentDB'.